

P1275R0

Desert Sessions: Improving hostile environment interactions

Published Proposal, 2018-10-06

This version:

<https://wg21.link/p1275r0>

Author:

[Isabella Muerte](#)

Audience:

LEWG, SG15, SG16

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Current Render:

[P1275R0](#)

Current Source:

slurps-mad-rips/papers/proposals/desert-sessions.bs

Implementation:

slurps-mad-rips/sessions

Abstract

Working with environment variables and command line arguments is painful, platform specific, and lacking a modern C++ touch.

Table of Contents

1 Revision History

1.1 Revision 0

2 Motivation

3 Design

3.1 Synopsis

3.1.1 Arguments

3.1.2 Environment

§ 1. Revision History

§ 1.1. Revision 0

Initial Release 🎉

§ 2. Motivation

Environment variables are an important aspect of modern systems. Yet, as of right now, C and C++ lack a standardized way to iterate through all current environment variables. An interface to iterate, get, set, erase, and modify environment variables, as well as providing familiar syntax found in other programming languages is needed.

Additionally, accessing command line arguments when `main` is outside of your control is currently not possible. Instead, objects must be initialized inside of `main`, or global variables must be initialized once `main` has started. Additionally, on Windows, the standard `char** argv` type is deprecated for use in `main`, and the only correct way to get the command line arguments is to call platform specific functions or use `WinMain` and `wmain`, both of which are non-standard entry points into the program. This is all such a strange approach compared to most other languages, especially due to command line arguments existing for the duration of a programs life. In languages such as Python, Rust, Ruby, Haskell, and C#, getting the list of command line arguments is just a function or member call away.

This paper aims to rectify both of these problems, while providing a familiar interface for users of `argv` as well as providing an interaction with Ranges.

§ 3. Design

This paper proposes the addition of a new header, `<session>`, for any and all types or functions that affect the current program's running session. To begin with we add two types into this header:

`std::arguments` and `std::environment`. Each one is a wrapper interface around the command line and environment variables respectively.

`std::arguments` attempts to meet *most* of the Named Requirements found in *SequenceContainer*. However, because it is an immutable container (it can not grow, shrink, delete or add its elements), its interface is effectively reduced to that of a `std::vector<T> const`. If mutation or modifications are desired, it is recommended that one construct a `std::vector` from the elements of `std::arguments`. The elements found inside of `std::arguments` is currently `std::string_view`. Whether the contents of these `string_views` are bytes or UTF-8 is intended to be implementation defined. The interface for `std::arguments` will need to be revisited once SG16's initial work is released, as a unicode-only interface for arguments is desirable.

`std::environment` is a bit more complex. It can be seen as an associative container, however it attempts to store no state unless absolutely needed. Additionally, for ease of use, it provides three ranges interfaces:

- `std::environment.keys()`
- `std::environment.values()`
- `std::environment`

Each of these ranges provides iteration over the keys, values, or the pair of both. Additionally subscript gets and sets are possible as well:

```
auto x = std::environment["PATH"];
```

The following straw polls are requested regarding the interface for both `std::arguments` and `std::environment`:

- Should they be `static inline` variables?
- Should they be templated on their argument values?
- Should they have (optionally template) aliases provided?
- Should they have shorter names available (`std::args`, `std::env`)?

Additionally, the plan is to provide two functions to easily iterate over a "path list". On platforms where this matters, an implementation specific path separator (e.g., `:` on POSIX, `;` on Windows) is used to separate multiple values when storing file system paths in environment variables. These functions create iterators that permit both the splitting of `std::string_view` objects into `std::filesystem::path` objects, as well as joining `std::filesystem::path` objects into a `std::string` that can then be assigned to an environment variable.

Lastly, on platforms where arguments or environment variables cannot typically be passed in or read from, attempting to iterate `std::arguments` and `std::environment` will yield an empty range, and attempting to index or subscript into them will return empty `std::string_view` objects.

Note: For security reasons we do not currently provide a cross platform way to get the current executable or home directory.

Wording has been withheld in this initial paper to see what interface changes are necessary.

§ 3.1. Synopsis

The `<session>` headers full specification is:

```
namespace std {
    class arguments;
    class environment;

    std::string join_paths (Iter begin, Iter end);
    std::string join_paths (Range);
}
```

§ 3.1.1. Arguments

The `arguments` class represents a mostly sequence-like container. It can be seen as a wrapper around `argv`, while still being accessible from outside of `main`. The object itself is immutable, and could technically be treated as a singleton. Reading from `arguments` is thread-safe, as is iterating over it. The type it currently returns is `std::string_view`, as this type can represent both UTF-8 and raw bytes. While the wording is not yet complete in this initial draft, it is intended that `arguments` only permit bytes from the operating system directly or UTF-8 only. Treating UTF-16 characters as bytes will not be permitted. At some point in the future, this will have to change, or an additional unicode-only `arguments` class will have to be implemented.

Additionally, it still provides access to the underlying `char** argv` and length of `argc` for backwards compatibility with APIs that still expect it as such.

```

class arguments {
    using iterator = /* implementation-defined */
    using reverse_iterator = reverse_iterator<iterator>;
    using value_type = std::string_view;
    using index_type = size_t;
    using size_type = size_t;

    value_type operator [] (index_type) const noexcept;
    value_type at (index_type) const noexcept(false);

    [[nodiscard]] bool empty () const noexcept;
    size_type size () const noexcept;

    iterator cbegin () const noexcept;
    iterator cend () const noexcept;

    iterator begin () const noexcept;
    iterator end () const noexcept;

    reverse_iterator crbegin () const noexcept;
    reverse_iterator crend () const noexcept;

    reverse_iterator rbegin () const noexcept;
    reverse_iterator rend () const noexcept;

    [[nodiscard]] char** argv () const noexcept;
    [[nodiscard]] int argc () const noexcept;
};

```

§ 3.1.2. Environment

The `environment` mapping object is also *mostly* immutable itself. It does support removing keys directly, although setting them directly is not supported. However, the environment can be modified by operating on the `variable` class it returns from `operator []`. This type acts as a proxy object and can be assigned to with nearly any string-like object. Additionally, it can return an object that has a pair of iterators that permit range operations to "split" on the path separator for a given environment variable.

Some platform's keys are case insensitive and for this reason, the `environment` silently checks both upper and lower case keys for their validity. This is safe to do as currently all platforms store their keys

as ASCII strings.

Note: Due to platform specific restrictions, environments can be iterated on while also actively mutating them. However, the underlying iterator's environment will still iterate on the unmodified version of the environment. After extensively looking at all operating systems available, it was confirmed that they all behave in this same way.

```

class environment {
    class variable {
        operator std::string_view () const noexcept;
        variable& operator = (std::string_view);
        std::string_view key () const noexcept;
        /* implementation-defined */ split () const;
    };

    using value_range = /* implementation-defined */
    using key_range = /* implementation-defined */
    using iterator = /* implementation-defined */
    using value_type = variable;

    template <class T>
    value_type operator [] (T const&) const; // see below

    value_type operator [] (std::string const&) const noexcept;
    value_type operator [] (std::string_view) const;
    value_type operator [] (char const*) const noexcept;

    template <class K>
    iterator find (K const&) const noexcept;

    bool contains (std::string_view) const noexcept;

    iterator cbegin () const noexcept;
    iterator cend () const noexcept;

    iterator begin () const noexcept;
    iterator end () const noexcept;

    size_type size () const noexcept;
    bool empty () const noexcept;

    value_range values () const noexcept;
    key_range keys () const noexcept;

    template <class K>
    void erase (K const&) noexcept;
};

```

All member functions that take a `T const&` follow the same rules regarding `std::string`'s 10th and 11th constructors regarding `std::string_view` conversion.

In the case of `environment::find`, this function follows the same rules of `std::map::find` as though it's `Compare::is_transparent` were valid.

Note: `environment` does not have a `Compare` template member, but can act as though it has one, since it works almost exclusively on strings.